BARODA UNIVERSITY
Vadodara | Gujarat | India
"Think Big... Think Beyond"

PROJECT REPORT FORMAT

JavaScript Framework (PBL) - C2620A3

BTech Semester 6

Academic Year 2026–27

Project Report Format

Instructions for Students

Prepare the report in Times New Roman, 12 pt, with 1.5 line spacing. Maintain the prescribed chapter sequence, clear headings, page numbers, figure/table captions, and consistent academic language.

Replace every bracketed placeholder with project-specific information. Include evidence of implementation, testing, deployment, and individual/team contribution.

BARODA UNIVERSITY
Vadodara | Gujarat | India
"Think Big... Think Beyond"

Cover Page

Project Title: [Enter Final Project Title]

Group Code: [Enter Group Code]

Submitted By: [Student Names and Enrollment Numbers]

Guided By: [Faculty Guide Name and Designation]

Course: JavaScript Framework (PBL) - C2620A3

Program: BTech Semester 6

Academic Year: Academic Year 2026–27

BARODA UNIVERSITY
Vadodara | Gujarat | India
"Think Big... Think Beyond"

Certificate

This is to certify that the project report entitled “[Project Title]” submitted by the students of BTech Semester 6 for the course “JavaScript Framework (PBL) - C2620A3” during Academic Year 2026–27 is a record of project work carried out under the guidance of the undersigned and submitted in partial fulfilment of the academic requirements of the course.

Guide: _____ Date: _____

Department/School: _____ Signature: _____

Declaration

We hereby declare that the project entitled “[Project Title]” is our original academic work prepared for the course “JavaScript Framework (PBL) - C2620A3”. All external sources, libraries, frameworks, documentation, and other materials used in the project have been appropriately acknowledged. The report and implementation have not been copied or submitted elsewhere for academic credit.

Acknowledgement

We express our sincere gratitude to our faculty guide, department, and institution for providing the guidance, resources, and support required to complete this project. We also acknowledge the official documentation and open-source communities whose technical resources supported our learning and implementation.

We thank all team members for their cooperation and contribution to requirements analysis, system design, implementation, testing, deployment, documentation, and presentation.

Abstract

This project presents a full-stack Single Page Application (SPA) developed using React with Vite for the frontend, Express.js for primary backend services, and MongoDB for persistent data storage. Redux is used for centralized client-side state management, React Router provides client-side navigation, and Axios is used for HTTP communication.

JWT-based authentication is implemented to secure protected application resources and manage authenticated sessions. A Python FastAPI microservice is included for modular backend functionality where a separate API service is beneficial. The project demonstrates the practical integration of modern frontend frameworks, REST APIs, NoSQL data management, token-based security, and cloud deployment.

Table of Contents

1. Introduction — Background, Problem Statement, Objectives, Scope

2. Literature Survey — Existing System, Limitations, Proposed System

3. System Requirements — Hardware, Software, Technology Justification

4. System Design — Architecture, ER Diagram & Schema, DFD, Use Case Diagram, API Endpoints

5. Implementation — Frontend, Redux/Axios, Express MVC/JWT, FastAPI, Mongoose

6. Testing — Strategy, Test Cases, API Testing

7. Deployment — Vercel/Netlify, Render/Railway, MongoDB Atlas, Environment Variables

8. Results and Screenshots

9. Conclusion and Future Scope

References

Appendix

Chapter 1 – Introduction

Background

Modern web applications increasingly use component-based frontend frameworks and API-driven backend services. React enables reusable UI components and SPA experiences, while Vite provides a fast development and build environment. Express.js and FastAPI support API-oriented backend development, and MongoDB provides flexible document-based persistence.

Problem Statement

State the real-world problem addressed by the project, target users, limitations of the existing workflow, required operations, and why a full-stack web application is appropriate.

Objectives

Develop a React/Vite SPA; manage state with Redux; implement navigation with React Router; communicate using Axios; develop Express.js APIs; implement JWT authentication; persist data using MongoDB/Mongoose; integrate a Python FastAPI microservice where required; and deploy the application.

Scope

Define users, modules, features, authentication boundaries, APIs, database operations, deployment environment, and future extensions. Clearly distinguish implemented features from proposed features.

Chapter 2 – Literature Survey

Existing System

Review comparable applications, conventional workflows, or existing solutions relevant to the project domain. Describe major features and technologies using reliable sources.

Limitations

Identify applicable limitations such as fragmented workflows, limited scalability, weak state management, inefficient navigation, lack of secure authentication, poor API separation, or insufficient centralized data management.

Proposed System

Present the proposed full-stack SPA using React/Vite, Redux, React Router, Axios, Express.js, MongoDB, JWT, and FastAPI, and explain how it addresses identified requirements.

Chapter 3 – System Requirements

Hardware Requirements

Modern multi-core CPU; 8 GB RAM minimum recommended; sufficient storage for source code and dependencies; reliable Internet connection for package installation, repositories, cloud services, and deployment.

Software Requirements

Frontend: React, Vite, Redux, React Router, Axios. Backend: Node.js, Express.js, MVC-oriented structure, JWT. Database: MongoDB with Mongoose. Microservice: Python FastAPI. Tools: VS Code, Git, GitHub, npm, Python, browser, Postman or equivalent. Deployment: Vercel/Netlify, Render/Railway, MongoDB Atlas.

Technology Justification

React/Vite provide modular SPA development. Redux supports centralized state management. React Router enables client-side navigation. Axios supports API communication and interceptors. Express.js provides a lightweight Node.js API layer. MongoDB/Mongoose support document storage and modeling. JWT enables token-based authentication. FastAPI supports isolated Python API functionality.

Chapter 4 – System Design

Architecture Diagram Explanation

Insert a labelled diagram showing React/Vite, Redux, React Router, Axios, Express.js MVC, JWT authentication, MongoDB/Mongoose, and Python FastAPI. Explain the data flow between components.

ER Diagram & Schema

Insert the project-specific data model. Provide collection names, fields, data types, validation rules, references, and indexes where applicable.

Data Flow Diagram

Provide context-level and/or Level-1 DFDs showing users, frontend processes, backend services, authentication, database operations, and FastAPI communication.

Use Case Diagram

Insert actors and major use cases such as registration, login, authentication, viewing data, permitted CRUD operations, and logout, adapted to the actual project.

API Endpoints

Document every implemented endpoint with Method, Endpoint, Purpose, Authentication, Request, Response, and Status Codes. Separate Express and FastAPI endpoints where appropriate.

Chapter 5 – Implementation

Frontend Implementation

Explain React/Vite structure, reusable components, pages, forms, validation, responsive UI, routing, and API communication.

Redux and Axios Interceptors

Explain Redux store/slices/actions or equivalent structure, state updates, Axios interceptors, authentication headers, and common response/error handling.

Express MVC and JWT Flow

Explain routes, controllers, services where applicable, Mongoose models, middleware, validation, error handling, login, JWT creation/verification, protected routes, and logout/token strategy where implemented.

FastAPI Microservice

Explain why the FastAPI service is separate, its responsibilities, endpoints, request/response models, validation, and integration with the main system.

Mongoose Database Integration

Describe MongoDB connection configuration, schemas/models, CRUD operations, relationships or references, validation, indexes, and error handling. Never include secrets.

Chapter 6 – Testing

Testing Strategy

Describe unit, integration, frontend, API, authentication, validation/error, database, and end-to-end testing as applicable. Explain comparison of expected and actual results.

Test Cases Table

Use columns: Test Case ID, Module, Scenario, Preconditions, Steps, Input, Expected Result, Actual Result, Status, Remarks.

API Testing

Document Postman or equivalent testing, including successful requests, validation failures, unauthorized requests, invalid/expired JWT cases, database cases, and response status codes.

Chapter 7 – Deployment

Frontend Deployment

Describe React/Vite deployment to Vercel or Netlify, including build command, output directory, SPA routing configuration, and environment variables.

Backend Deployment

Describe Express.js and FastAPI deployment to Render or Railway, including build/start commands, service configuration, CORS, health checks, and environment variables.

MongoDB Atlas

Explain cluster, database user, network access, connection string, collections, and production configuration. Never publish credentials.

Environment Variables

List variable names and purposes without values, including database URL, JWT configuration, API base URLs, and FastAPI service URL. Keep secrets outside source control.

Chapter 8 – Results and Screenshots

Present the final working application with screenshots of major modules: landing/login/registration, dashboard, core features, forms, validation/error messages, authenticated navigation, API results, responsive views, and deployed pages. Give every screenshot a figure number and descriptive caption.

Chapter 9 – Conclusion and Future Scope

Conclusion

Summarize how the implemented system satisfies the objectives, how the architecture works, the effectiveness of JWT authentication, and the major learning outcomes.

Future Scope

Possible extensions include role-based access control, refresh-token rotation, automated testing and CI/CD, stronger logging/observability, caching, containerization, Kubernetes deployment, security hardening, performance optimization, accessibility improvements, and additional microservices.

References

Use a consistent academic reference style. Cite official documentation and scholarly/technical sources used in the project, including relevant React, Vite, Redux, React Router, Axios, Express.js, MongoDB/Mongoose, JWT, Python, FastAPI, Vercel/Netlify, Render/Railway, and MongoDB Atlas documentation.

Appendix

Appendix A – GitHub Repository: [Insert Repository URL]

Appendix B – Live Frontend URL: [Insert URL]

Appendix C – Live Backend/API URL: [Insert URL]

Appendix D – FastAPI Service URL: [Insert URL, if separately deployed]

Appendix E – API Collection/Documentation: [Insert URL or attach export]

Appendix F – GitHub Commit History Screenshots: Attach screenshots showing meaningful individual/team contributions.

Final Submission Instructions

1. All group members must contribute meaningfully to the project and documentation.

2. The report must accurately represent implemented work; do not claim features that were not developed.

3. Plagiarism, copied reports/source code without attribution, fabricated test results, or misleading screenshots are not acceptable.

4. Submit the project and report before the prescribed deadline.

5. Replace all placeholders before final submission.

6. Verify GitHub links, live links, screenshots, API documentation, and deployment details.

7. Attach GitHub commit-history screenshots demonstrating individual/team contribution.

8. Do not include passwords, JWT secrets, database credentials, API keys, or confidential environment-variable values.

9. Proofread the final report and ensure figures, tables, diagrams, references, and page numbers are correctly labelled.